

UK Tax & Shipping Shopify Custom App

Trincos

Build Steps, Configuration & Full System Flow

Prepared By: **Titus Senthilkumaran**

1. Overview & Purpose

This document explains how to build, deploy, and configure the UK Tax and Shipping Custom App for Shopify. The app replaces Shopify's static shipping rate tables with a dynamic calculation engine that determines shipping cost per order based on postcode zone, product weight, supplier (source warehouse), and product type — while also handling tax-only line items (software, warranties, services) that should never carry a shipping charge.

The underlying business logic (zoning, weight fallback, rate bands, heavy-weight surcharge, and tax-only handling) is defined in the companion document "UK Shipping System — Full Technical Documentation." This guide covers the Shopify-side implementation: how that logic is wrapped into a Custom App, how the app is registered and configured inside Shopify, and how requests flow end-to-end at checkout.

Why a Custom App (not a public app)

Because this logic is specific to one merchant (Trincos-style multi-warehouse dropshipping via Ingram Micro and Tech Data), it is built and installed as a Shopify Custom App scoped to a single store, rather than distributed via the Shopify App Store.

2. What Is a Shopify App?

A Shopify app is an external piece of software that connects to a Shopify store through Shopify's APIs, extending what the store can do beyond its built-in features. Apps authenticate against a store using an access token and a defined set of API scopes (permissions), and communicate with Shopify over REST/GraphQL Admin APIs, webhooks, and — for this project — the Carrier Service API used for shipping rate calculation.

Shopify apps fall into three broad categories:

- Public apps — listed on the Shopify App Store, installable by any merchant, built and managed through a Shopify Partner organization.
- Custom apps — built for and installed on one specific store only, created directly from that store's Admin. This is the category this project falls into.
- Custom distribution apps — a variant of a Partner-built app installed on a small, named set of stores without going through the public App Store listing process.

A Custom App is the right fit here because the tax/shipping logic is bespoke to Trincos's own supplier setup (Ingram Micro and Tech Data) and UK zone structure — it isn't meant to be reused by other merchants. Once created, a Custom App behaves like any other Shopify app from the store's point of view: it gets its own access token, its own scopes, and Shopify sends it API requests (in this case, shipping rate requests at checkout) the same way it would for an installed public app.

3. Architecture Summary

The app is composed of three cooperating pieces:

1. Shopify Store — sends carrier-service shipping rate requests at checkout, and (optionally) order/tax webhooks for reconciliation.
2. Custom App Backend (Node.js) — deployed as serverless functions on Vercel, exposing a publicly reachable HTTPS API that implements the zoning, weight, and rate logic, and returns calculated rates or tax adjustments.
3. Data Layer (Supabase / Postgres) — stores `shipping_rules_UK` (rate bands per zone), postcode-prefix-to-zone mapping, and per-product overrides (weight, tax-only flag, product type).

High-level diagram: *Checkout → Shopify Carrier Service call → Vercel-hosted API function → Rules DB lookup (Supabase) → Calculated rate(s) → Response → Shopify displays rate to customer.*

4. Step-by-Step: Building the Custom App

4.1 Create the Custom App in Shopify Admin

4. In Shopify Admin, go to Settings → Apps and sales channels → Develop apps.
5. Click Allow custom app development (first time only), then Create an app.
6. Name the app, e.g. "Trincos UK Tax & Shipping Engine."
7. Under Configuration, set the Admin API scopes needed (see 4.2).
8. Click Install app to generate the Admin API access token — store this token securely (environment variable, never in source control).

4.2 Define Required API Scopes

Scope	Purpose
read_shipping	Read existing shipping zones/rates for validation during setup
write_shipping	Register and manage the Carrier Service
read_orders	Read order line items to re-verify tax-only vs. shippable items
read_products	Read product weight and product_type metafields
write_order_edits (optional)	Adjust shipping/tax lines post-checkout if reconciliation is needed

4.3 Scaffold the Backend Service

The backend exposes two responsibilities: a Carrier Service endpoint (for shipping rate calculation) and an admin dashboard for managing rates, mappings, and connections. The actual deployed project uses the Shopify App template — React Router with flat file-based routing, Prisma for the database layer, Supabase for product data, and Polaris for the admin UI — rather than a bare serverless scaffold. See Section 11 for the complete, real file-by-file structure.

4.4 Implement the Carrier Service (Shipping) API

Shopify calls this endpoint at checkout with the customer's address and cart contents; the app must respond within Shopify's timeout window (10 seconds) with one or more shipping rate options. In the actual codebase this is `app/routes/app.api.shipping-rates.tsx` — its action handler receives the POST request, fetches product data, runs `AdvancedShippingEngine`, and returns the rate response (see Section 11).

POST `https://<your-vercel-project>.vercel.app/app/api/shipping-rates`

Request body shape sent by Shopify (simplified):

```
{
  "rate": {
    "destination": { "postal_code": "SW1A 1AA", "country": "GB" },
    "items": [
      { "grams": null, "properties": { "source_type": "COMMON(INGRAM)", "product_type":
"Servers" } }
    ]
  }
}
```

The app must translate this into the internal input structure (`items[]` with `weight`, `source_type`, `product_type`, plus `postcode`), then run it through the calculation pipeline described in Section 7, and return:

```
{
```

```
"rates": [
  { "service_name": "Trincos UK Delivery", "service_code": "TRINCOS_UK",
    "total_price": "1800", "currency": "GBP", "description": "Zone 1 · Standard" }
]
```

Units note

Shopify's `total_price` for carrier services is in the smallest currency unit (pence for GBP), so £18.00 must be sent as "1800".

4.5 Implement the Tax-Only Handling

Before rate calculation runs, each line item is checked against the tax-only rule (Section 7.6). If a product is flagged tax-only (software, warranty, service), it is excluded from the shipment grouping entirely so it never contributes weight or triggers a rate — its shipping contribution is forced to zero.

4.6 Register the Carrier Service with Shopify

Registration is a one-time Admin API call that tells Shopify where to send rate requests. In the actual app this is wrapped by `app/lib/carrierService.ts` and exposed through the `app/routes/app.carrier-service.tsx` route, which the admin Connection panel (`ConnectionPanel.tsx`) calls with a single click — rather than requiring a manual API request each time:

```
POST /admin/api/2025-01/carrier_services.json
{
  "carrier_service": {
    "name": "Trincos UK Tax & Shipping Engine",
    "callback_url": "https://<your-vercel-project>.vercel.app/app/api/shipping-rates",
    "service_discovery": true
  }
}
```

After registration, go to Settings → Shipping and delivery → Shipping profiles, edit the relevant profile's rate zones, and enable the new carrier service so it appears as a rate provider alongside (or instead of) flat rates.

4.7 Deploy to Vercel

- Connect the repository to a Vercel project (Vercel Dashboard → Add New → Project → import from Git).
- Vercel auto-detects the `api/` folder and deploys each file inside it as a serverless function — no separate server process or port to manage.
- Set the environment variables listed in Section 5.1 (`DATABASE_URL`, `DIRECT_URL`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SHOPIFY_API_KEY`, `SHOPIFY_API_SECRET`, `SCOPES`, `SHOPIFY_APP_URL`) in Vercel Project Settings → Environment Variables, applied per environment (Production / Preview / Development).
- Every push to main triggers a Production deployment automatically; every pull request gets its own Preview deployment URL, useful for testing rate changes before they go live.
- HTTPS and a `*.vercel.app` domain are provided automatically; a custom domain/subdomain can be attached under Project Settings → Domains if preferred for the `callback_url`.
- Add a `/api/health` endpoint and attach it to an uptime monitor — Shopify will silently fall back to default rates if the carrier service times out or errors, so downtime is easy to miss without monitoring.

4.8 Test in Checkout

9. Use a development/staging store or Shopify's checkout in test mode.
10. Add products with different `product_type` and `source_type` combinations to the cart.

11. Enter test postcodes covering each zone (e.g. SW1A 1AA for Zone 1, IV1 1AA for Zone 2, BT1 1AA for Zone 3, JE1 1AA for Zone 4).
12. Confirm the returned rate matches manual calculation from Section 7.
13. Test a multi-supplier cart (one Ingram item, one TechData item) and confirm two shipments are costed separately, then summed.
14. Test a tax-only product alone in the cart and confirm £0 shipping / no shipping line blocking checkout.

5. App Configuration

5.1 Environment Variables

These are the actual environment variables configured on the Vercel project (Project Settings → Environment Variables). Values are never committed to source control — set each one as a placeholder locally, then paste the real secret directly into Vercel.

Variable	Description	Environment
DATABASE_URL	Pooled Postgres/Supabase connection string used by Prisma at runtime — placeholder: <your-database-connection-string>	All Environments
DIRECT_URL	Direct (non-pooled) Postgres connection string, used by Prisma for migrations — placeholder: <your-direct-connection-string>	All Environments
SUPABASE_URL	Base URL of the Supabase project — placeholder: <your-supabase-project-url>	All Environments
SUPABASE_SERVICE_ROLE_KEY	Supabase service-role key used by app/supabase.server.ts for admin-level DB access — placeholder: <your-supabase-service-role-key>	All Environments
SHOPIFY_API_KEY	Client ID of the Shopify app, from the Partner Dashboard / app config — placeholder: <your-shopify-api-key>	All Environments
SHOPIFY_API_SECRET	Client secret of the Shopify app — placeholder: <your-shopify-api-secret>	All Environments
SCOPES	Comma-separated OAuth scopes requested on install, e.g. read_products,write_shipping — placeholder: <your-oauth-scopes>	All Environments
SHOPIFY_APP_URL	Public base URL the app is reachable at (the Vercel Production domain) — placeholder: <your-production-app-url>	Production

5.2 Shipping Rules Table (shipping_rules_UK)

Stored in Supabase and read by the rate engine on every request (cached briefly to reduce DB load). Zone 1 example, as defined in the source specification:

Weight Band	Zone 1 Rate
0 – 2 kg	£5
2 – 5 kg	£8
5 – 10 kg	£12
10 – 20 kg	£18

Weight Band	Zone 1 Rate
20 – 30 kg	£28
> 30 kg	£28 + (extra kg × £0.50)

Zones 2–4 follow the same band structure with their own rate tables and are configured the same way in the database; only Zone 1 is reproduced above as the reference example.

5.3 Postcode-to-Zone Mapping

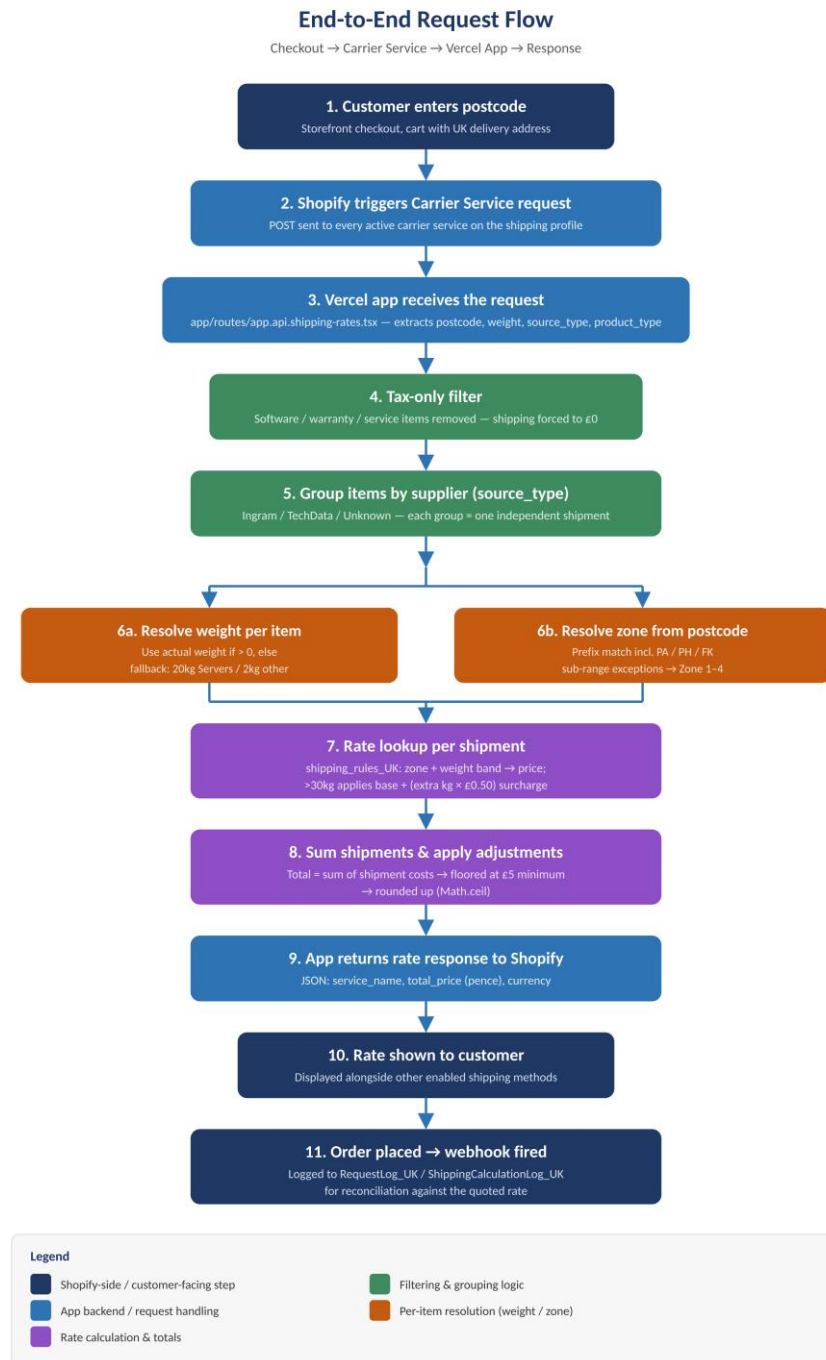
Prefix(es)	Zone
IV, HS, KW, ZE, AB	ZONE_2 — Highlands & Remote
BT	ZONE_3 — Northern Ireland
IM, JE, GY	ZONE_4 — Offshore Islands
PA1–PA19	ZONE_1 (exception within PA range)
PA20+	ZONE_2
PH1–PH7	ZONE_1 (exception within PH range)
PH8+	ZONE_2
FK	ZONE_1
All others	ZONE_1 — Mainland UK

5.4 Product & Supplier Configuration

- `product_type` on each product (or its metafield) determines the default weight fallback (Servers → 30kg, all other types → 2kg) and whether it is tax-only.
- `source_type` is set per product/variant to identify the fulfilling supplier: INGRAM, TECHDATA, or the bracketed common-stock forms COMMON(INGRAM) / COMMON(TECHDATA).
- Missing or unrecognized `source_type` is normalized to "unknown" and still forms its own shipment group rather than being dropped.

6. Full Request/Response Flow (End to End)

The diagram below traces one checkout request from the moment a customer enters a postcode through to order reconciliation. The numbered steps that follow give the same flow in text.



15. Customer reaches checkout on the storefront and enters a delivery postcode.
16. Shopify checkout triggers a carrier-service rate request to every active carrier service on the shipping profile, including this app.
17. App backend receives the request and extracts: destination postcode, and per line item — weight (if present), source_type, product_type.
18. Tax-only filter runs first: any line item flagged tax-only is removed from shipment consideration (Section 7.6).
19. Remaining items are normalized by source_type (bracket extraction + lowercase) and grouped by supplier (Section 7.1–7.2).
20. For each supplier group, weight is resolved per item — actual weight if valid, else the product-type default (Section 7.3).
21. Postcode is mapped to a zone using the prefix rules, including the PA/PH sub-range exceptions (Section 7.4).

22. For each supplier group, the group's total weight and the destination zone are used to look up the matching rate band; weights above 30kg apply the heavy-weight surcharge formula (Section 7.5, 7.7).
23. Per-shipment costs are summed into a total shipping figure, floored at the £5 minimum, then rounded up to the nearest whole currency unit (Section 7.9).
24. The app returns one (or more) rate objects to Shopify in the carrier-service response format.
25. Shopify displays the returned rate(s) to the customer at checkout alongside any other enabled shipping methods.
26. On order completion, Shopify fires order webhooks; these can optionally be consumed by the app for logging/reconciliation against the rate that was actually charged.

7. Core Calculation Logic (Reference)

This section restates the calculation rules from the source specification as implemented inside the app's rate engine.

7.1 Source Type Normalization

source_type is normalized by extracting the value inside brackets when present, then lowercasing it. Example: COMMON(INGRAM) → ingram; TECHDATA → techdata; missing/null → unknown.

7.2 Grouping by Supplier

Products are grouped by normalized source_type. Each group represents an independent shipment, because each supplier ships separately from its own warehouse.

7.3 Weight Resolution

Actual product weight is used when valid; otherwise a product-type default fallback applies — 30kg for Servers, 2kg for all other product types. Null weights are intentionally passed through to the engine rather than overridden earlier in the pipeline, so fallback logic stays centralized.

7.4 Postcode-to-Zone Mapping

Four zones exist: ZONE_1 (Mainland UK), ZONE_2 (Highlands & Remote), ZONE_3 (Northern Ireland), ZONE_4 (Offshore Islands), resolved via prefix matching with PA/PH sub-range exceptions as listed in Section 5.3.

7.5 Rate Lookup

Rates are retrieved from shipping_rules_UK using $\text{min_weight} \leq \text{weight} < \text{max_weight}$ matching against the resolved zone's band table.

7.6 Tax-Only Handling

Tax-only products (software, warranties, services) never generate a shipping charge — if a line item is tax-only, shipping for that item is forced to 0 and it is excluded from shipment grouping.

7.7 Heavy-Weight Surcharge (> 30kg)

basePrice = the 20–30kg band rate; extraWeight = weight – 30; surcharge = extraWeight × £0.50; finalRate = basePrice + surcharge. Example: 86kg → £56 final charge (£28 base + 56kg extra × £0.50).

7.8 Per-Shipment and Total Calculation

Each supplier group's cost is computed independently via the rate lookup (with surcharge where applicable), then all shipment costs are summed to produce totalShipping.

7.9 Final Adjustments

$\text{finalShipping} = \max(\text{calculatedShipping}, £5)$ — a £5 floor is always applied — and the result is rounded up with `Math.ceil` before being returned.

7.10 Output Structure

```
{
  "success": true,
  "basePrice": 2586.72,
  "taxAmount": 517.34,
  "carrierCharge": 56,
  "total": 3160.06
}
```

8. Worked Examples

8.1 Single-Supplier Example

Postcode SW1A 1AA · Product Type: Servers · Weight: null · Source: Ingram.

- Source normalizes to "ingram".
- Weight falls back to 30kg (Servers default, since weight was null).
- Postcode SW1A 1AA maps to ZONE_1.
- 20kg in ZONE_1 falls in the 10–20kg band → £18.

8.2 Multi-Supplier Example

Cart contains: a Server from Ingram (null weight), and a Rack from TechData (65kg).

- Ingram group: 20kg fallback → ZONE_1 rate → £18.
- TechData group: 65kg → exceeds 30kg → heavy-weight formula applies (base £28 + surcharge). See Appendix A.8 for the source specification's exact reference figure and a formula-vs-example discrepancy note.
- Total shipping = sum of both shipment costs, then floored at £5 minimum and rounded up.

9. Error Handling & Fallbacks

Condition	Fallback Behaviour
Unknown/missing source_type	Normalized to "unknown"; still forms its own shipment group
Missing/invalid weight	Product-type default weight applied (30kg Servers / 2kg other)
Postcode prefix not mapped	Defaults to ZONE_1 (Mainland UK)
Weight exceeds all defined bands (>30kg)	Heavy-weight surcharge formula applied instead of a fixed band
Calculated shipping below £5	Floored to the £5 minimum charge
Carrier service request times out (>10s)	Shopify falls back to other active shipping methods on the profile — monitor uptime to catch this

10. Deployment & Go-Live Checklist

- Custom App created in Shopify Admin with correct scopes and access token stored securely.
- Backend deployed on Vercel, environment variables set for the Production environment, with health-check monitoring against the Vercel deployment URL.
- `shipping_rules_UK` and `postcode-zone` tables populated in Supabase for all four zones.

- Carrier Service registered via Admin API with `callback_url` pointing at the Vercel Production domain, and enabled on the correct shipping profile(s).
- Test orders run across all four zones, a heavy-weight (>30kg) item, a multi-supplier cart, and a tax-only cart.
- Logging/reconciliation in place to compare quoted vs. charged shipping over the first weeks post-launch.
- Rollback plan: disable the carrier service in the shipping profile to instantly revert to Shopify's default flat rates if an issue is found.

11. Codebase Reference — Actual Implementation

This section documents the real project structure as implemented, for anyone maintaining or extending the app. It's a Shopify App template project — React Router with file-based routing, Prisma as the ORM, Supabase for product data, and Polaris for the embedded admin UI — deployed on Vercel.

11.1 Core / Server Setup

File	Purpose
<code>app/db.server.ts</code>	Singleton Prisma client. Reuses a global instance in development to avoid connection leaks; creates a new one in production.
<code>app/shopify.server.ts</code>	Configures the Shopify app: API key/secret, API version, OAuth scopes, Prisma-backed session storage, auth path prefix, distribution. Exports <code>authenticate</code> , <code>registerWebhooks</code> , <code>sessionStorage</code> .
<code>app/supabase.server.ts</code>	Singleton Supabase client factory. Lazily creates an admin client from <code>SUPABASE_URL</code> / <code>SUPABASE_SERVICE_ROLE_KEY</code> , session persistence disabled.
<code>app/routes.ts</code>	Configures React Router to use flat routes, auto-discovered from <code>app/routes/</code> .
<code>app/entry.server.tsx</code>	Server-side React rendering entry point. Streams SSR HTML, handles bot vs. browser hydration, sets Shopify's document response headers.
<code>app/root.tsx</code>	Root React component — HTML shell (head, meta, fonts, Outlet, scroll restoration) shared across all routes.

11.2 Auth Routes(Template default files)

File	Purpose
<code>app/routes/auth.\$.tsx</code>	Catch-all route for Shopify OAuth callbacks. Delegates to <code>authenticate.admin()</code> .
<code>app/routes/auth.login/route.tsx</code>	Login page where merchants enter their shop domain to start OAuth.
<code>app/routes/auth.login/error.server.tsx</code>	Translates Shopify login errors (missing/invalid shop) into user-facing messages.
<code>app/routes/_index/route.tsx</code>	Public landing page shown before login; redirects to <code>/app</code> once a shop domain is provided.

11.3 App Shell & Admin Dashboard

File	Purpose
app/routes/app.tsx	Parent layout for all /app pages. Requires admin auth, wraps children in Polaris AppProvider + AppNav, handles saving rate defaults (tax rate, carrier charge, USD-GBP rate).
app/routes/app._index.tsx	Main admin dashboard. Handles product sync actions (start/manual/cancel/resume) and loads data for every admin panel: product counts, mappings, shipping logs, request logs, carrier service status, rate settings.
app/routes/app.additional.tsx	Placeholder page demonstrating multi-page app navigation.

11.4 API / Backend Routes

File	Purpose
app/routes/app.api.shipping-rules.tsx	Lists and saves shipping zone/weight-based rate rules. loader returns rules + zone prefixes; action replaces the rule set.
app/routes/app.api.shipping-rates.tsx	The Carrier Service webhook Shopify calls at checkout. action (POST) fetches product data, runs AdvancedShippingEngine, converts prices, logs the request, and returns the rate response.
app/routes/app.calculate-shipping.tsx	Powers the admin calculator UI — takes SKUs + postcode, runs calculateShippingForSkus, logs and returns the breakdown.
app/routes/app.carrier-service.tsx	Registers, fetches, and deletes the shop's Shopify carrier service; used by the Connection panel.
app/routes/api.start-sync.ts	Vercel cron job — creates "running" sync jobs for all known shops, preventing duplicates.
app/routes/api.recover.ts	Vercel cron job (every minute) — resumes any "running" or "failed" sync jobs that were interrupted.

11.5 Shared Libraries (app/lib/)

File	Purpose
app/lib/AdvancedShippingEngine.ts	Core rate calculation engine: resolves zone from postcode, groups items by supplier source type, calculates total weight with fallbacks, looks up DB rates by zone/weight tier, applies the >30kg surcharge.
app/lib/shippingCalculator.ts	High-level calculator. Fetches shop settings + product data, determines tax-only status, calls AdvancedShippingEngine, applies the minimum charge, returns the full breakdown. Supports single- and multi-SKU calls.
app/lib/carrierService.ts	Helpers for the Shopify Carrier Service API: fetch existing services, register a new one with callback URL, delete one, cache the result locally.
app/lib/productSync.ts	Manages product sync jobs between the source (Supabase/Shopify products) and ProductMapping_UK. Batches of 100, cursor-tracked, cancel/resume support, cron-driven.
app/lib/requestLog.ts	Logs incoming/outgoing HTTP requests to RequestLog_UK. Provides logRequest() and getRecentLogs().

11.6 Admin Components (app/components/)

File	Purpose
app/components/AppNav.tsx	Client-side nav bar inside the Shopify admin bar, with links to Home and Additional page.
app/components/admin/types.ts	Shared TypeScript interfaces: ProductRow, MappingRow, LogRow, ConnectionInfo, CarrierServiceInfo, RateSettings, ShippingCalculationResult, RequestLogEntry.
app/components/admin/DataTables.tsx	Shows product counts, mapping stats, sync job status with a progress bar, and sync control buttons.
app/components/admin/LogsPanel.tsx	Paginated table of recent shipping calculation logs with status badges.
app/components/admin/ConnectionPanel.tsx	Manages the carrier service connection — shows the callback URL/status, register/remove controls, a local test callback tool.
app/components/admin/ShippingCalculator.tsx	Interactive tool to calculate shipping for one or more SKUs, with optional postcode override and a detailed cost breakdown.
app/components/admin/RequestLogsPanel.tsx	Table of the last 10 HTTP requests — type, method, endpoint, status code, duration, timestamp.
app/components/admin/RateSettings.tsx	Form for default tax rate, carrier charge, and USD-to-GBP rate; saves via the app.tsx route action.
app/components/admin/ShippingRulesPanel.tsx	Editable table for zone/weight rate rules — add/remove rows, inline edit, save to the API.

11.7 Root Configuration Files

File	Purpose
package.json	Project definition (shipping-calculation-uk), npm scripts (dev/build/start/lint/typecheck), dependencies (React Router, Shopify app SDK, Prisma, Supabase), Node engine requirement.
shopify.app.toml	Shopify app descriptor — name, URL, embedded mode, API version, OAuth scopes (orders, products, shipping), webhook subscriptions, auth redirect URLs.
shopify.web.toml	Tells the Shopify CLI this is a React Router web app; configures predev/dev commands (Prisma generate + migrate + dev server).
vite.config.ts	Vite config — React Router plugin, tsconfig path aliases, HMR settings, allowed hosts, dependency optimization for @shopify/app-bridge-react.
tsconfig.json	TypeScript settings — strict mode, ES2022 target, JSX react runtime, bundler module resolution.
vercel.json	Vercel deployment config — defines the scheduled cron jobs hitting /api/start-sync (daily) and /api/recover (every minute).
.shopify/project.json	Shopify CLI project metadata mapping the app client ID to a dev store URL.

Not documented here: tsconfig.tsbuildinfo — a TypeScript-generated incremental build cache with no manual-maintenance relevance.

11.8 Database Schema (prisma/schema.prisma)

Defines every table backing the app: Session (Shopify sessions), Settings_UK (per-shop tax/rate config), shopify_products_final_UK (product data), ProductMapping_UK (SKU-to-Ingram mapping), ProductSyncJob_UK (sync job tracking), shipping_rules_UK / shipping_zone_prefixes (the rate tables referenced throughout this guide), RequestLog_UK / ShippingCalculationLog_UK (logging), CarrierService (carrier service registry), plus the Supabase auth/inventory models.

Appendix A: Key System Data (Reference)

This appendix consolidates the exact input/output schemas, formats, and reference tables from the source specification, for quick lookup by whoever maintains the rate engine.

A.1 Input Structure

```
items: [
  {
    weight: number | null,
    source_type: string,
    product_type: string
  }
]
postcode: string
```

A.2 Source Type — Supported Formats & Normalization

Raw source_type	Normalized
INGRAM	ingram
TECHDATA	techdata
COMMON(INGRAM)	ingram
COMMON(TECHDATA)	techdata
null / missing	unknown

Normalization rule: extract the value inside brackets if present, then lowercase the result.

A.3 Grouping Structure

Products are grouped into independent shipments by normalized source_type:

```
[
  [ ingram items ],
  [ techdata items ],
  [ unknown items ]
]
```

A.4 Weight Defaults (Fallback)

Product Type	Default Weight
Servers	30 kg

Product Type	Default Weight
Other Products	2 kg

Rule: if weight exists and is greater than 0, use it; otherwise apply the default for that product_type. Null weights are passed through to the engine rather than overridden earlier, so the fallback stays centralized in one place.

A.5 Zone Reference

Zone	Description
ZONE_1	Mainland UK
ZONE_2	Highlands & Remote
ZONE_3	Northern Ireland
ZONE_4	Offshore Islands
Prefix	Zone
IV, HS, KW, ZE, AB	ZONE_2
BT	ZONE_3
IM, JE, GY	ZONE_4
All others	ZONE_1

Special cases (numeric sub-ranges within a prefix):

PA:

PA1-PA19 -> ZONE_1
PA20+ -> ZONE_2

PH:

PH1-PH7 -> ZONE_1
PH8+ -> ZONE_2

FK:

Always -> ZONE_1

Worked example: postcode PH15 → prefix PH, number 15 → since 15 falls in the PH8+ range → ZONE_2.

A.6 Rate Table — ZONE_1 (Reference)

Weight Range	Price
0-2 kg	£5
2-5 kg	£8
5-10 kg	£12
10-20 kg	£18
20-30 kg	£28

Matching logic: $\text{min_weight} \leq \text{weight} < \text{max_weight}$, looked up against the shipping_rules_UK table for the resolved zone.

A.7 Heavy-Weight Formula (> 30 kg)

```
basePrice    = price for 20-30kg band
extraWeight  = weight - 30
surcharge    = extraWeight × 0.5
```

```
finalRate = basePrice + surcharge
```

Worked example: weight = 86kg → base = £28, extra = 56kg × £0.50 = £28, final = £56.

A.8 Per-Shipment & Total Calculation

Supplier	Weight	Cost
Ingram	20 kg	£18
TechData	65 kg	£46

shipmentCost = getRate(zone, shipmentWeight), computed independently per supplier group; totalShipping = sum(all shipment costs).

A.9 Final Adjustments

```
finalShipping = max(calculatedShipping, £5) // £5 minimum floor
totalShipping = Math.ceil(totalShipping) // round up
```

A.10 Tax-Only Handling

Certain product types are tax-only — no physical delivery, no shipping charge. Examples: software, warranties, services.

```
if taxOnly: shipping = 0
```

A.11 Output Structure

```
{
  "success": true,
  "basePrice": 2586.72,
  "taxAmount": 517.34,
  "carrierCharge": 56,
  "total": 3160.06
}
```

A.12 Complete End-to-End Example

Input: postcode SW1A 1AA · product_type Servers · weight null · source Ingram.

27.Source → ingram

28.Weight → null → fallback → 30 kg

29.Zone → ZONE_1

30.Rate → 30kg → £28

Output: Shipping = £28.

A.13 Multi-Supplier Example

Input: Server (Ingram, null weight) + Rack (TechData, 65kg).

- Ingram → 20kg → £18
- TechData → 65kg → £28 base + surcharge (per A.7/A.8 note above)
- Total = sum of both shipment costs